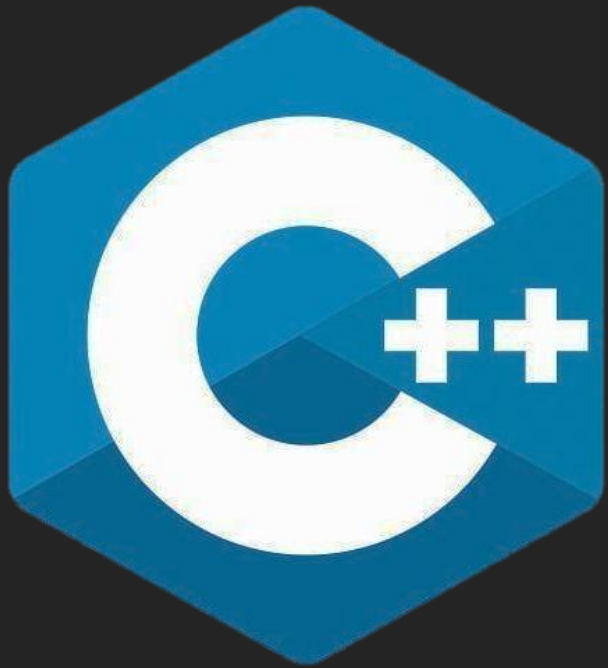




Cs50
Arrays



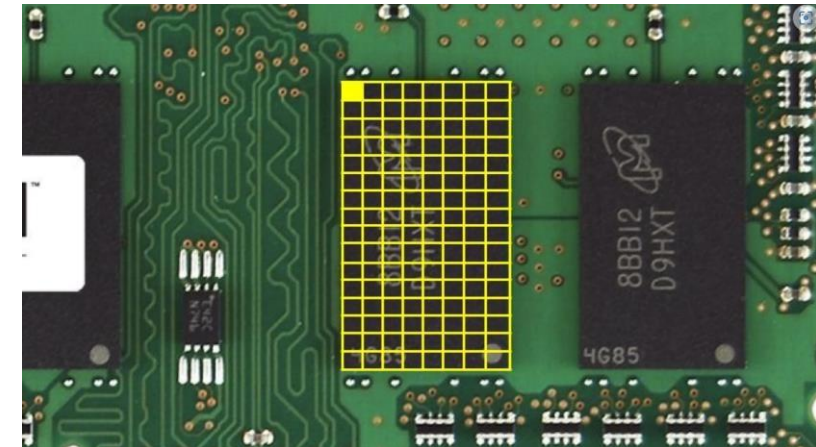
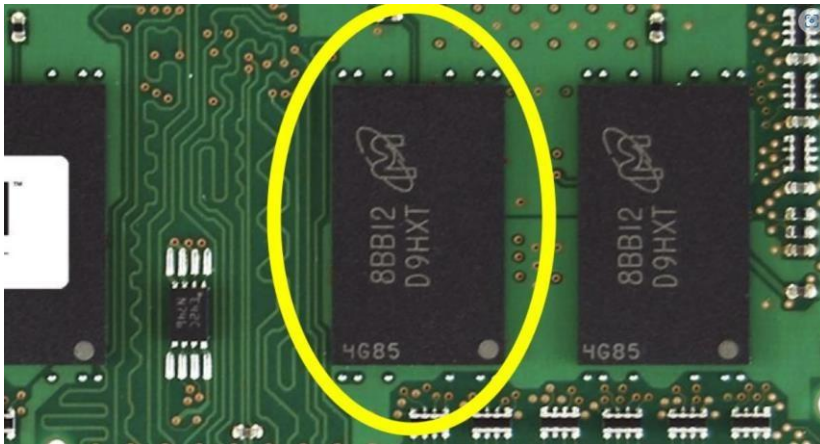
➤ Arrays



Array

- In Week 0, we talked about *data types* such as `int` , `char` , `float` , `string` ,.....
- Each data type requires a certain amount of system resources
- Inside of your computer, you have a finite amount of memory available.
- Physically, on the memory of your computer, you can imagine how specific types of data are stored on your computer. You might imagine that a `char` , which only requires 1 byte of memory, may look as follows:

```
bool    1 byte
int      4 bytes
long     8 bytes
float    4 bytes
double   8 bytes
char     1 byte
string   ? bytes
...
```



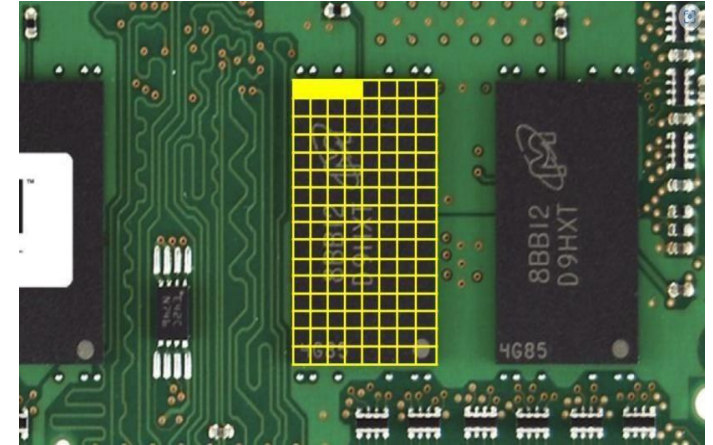
Array

- Similarly, an **int**, which requires 4 bytes might look as follows:

```
// Scores  
int score1 = 72;  
int score2 = 73;  
int score3 = 33;
```



72 score1				73 score2			
33 score3							



Array

The general form of initialization of array is,

dataType arrayName[arraySize] = { list of values };

```
mark[0]=10;  
mark[1]=20;  
mark[2]=30;  
mark[3]=40;  
mark[4]=50;  
mark[5]=60;
```

int mark[5]={10,20,30,40,50,60};

10	20	30	40	50	60
mark[0]	mark[1]	mark[2]	mark[3]	mark[4]	mark[5]

Array

```
int hours[5];  
hours[0] = 7;  
hours[1] = 9;
```

hours

7	9	?	?	?
hours[0]	hours[1]	hours[2]	hours[3]	hours[4]

Array

```
int hours[5] = {7, 9, 8, 7, 8};
```

hours

7	9	8	7	8
hours[0]	hours[1]	hours[2]	hours[3]	hours[4]

You are given an array {10, 20, 30 ,40, 50, 60, 70}

Note : the array is in ascending order.

A program that arranges numbers in ascending order.



```
1  #include<iostream>
2
3  using namespace std;
4
5
6  int main(){
7
8      int array[7] = {10, 20, 30, 40 ,50 , 60, 70};
9
10     cout << "Numbers in ascending order: ";
11
12     for(int i = 0; i < 7; i++){
13         cout << array[i] << " ";
14     }
15
16     cout << "\n";
17
18     return 0;
19 }
20
```





Write a program that arranges numbers in descending order.



You are given an array {10, 20, 30 ,40, 50, 60, 70}

Note : the array is in ascending order.

A program that arranges numbers in descending order.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6
7      int array[7] = {10, 20, 30, 40, 50, 60, 70};
8
9      cout << "Numbers in descending order: ";
10
11     for (int i = 6; i >= 0; i--) {
12         cout << array[i] << " ";
13     }
14
15     cout << "\n";
16
17     return 0;
18 }
```



```
#include <iostream>
using namespace std;

int main(void)
{
    // Get scores
    int scores[3];
    scores[0] = 72;
    scores[1] = 73;
    scores[2] = 33;

    // Print average
    cout << "Average: " << (scores[0] + scores[1] + scores[2]) / 3.0;
}
```



```
#include <iostream>
using namespace std;

int main(void)
{
    // Get scores
    int scores[3];
    cin >> scores[0];
    cin >> scores[1];
    cin >> scores[2];

    // Print average
    cout << "Average: " << (scores[0] + scores[1] + scores[2]) / 3.0);
}
```



You can see how while the above code works, there is still an opportunity for improving our code. Revise your code as follows:

```
#include <iostream>
using namespace std;

int main(void)
{
    // Get scores
    int scores[3];
    for(int i = 0; i < 3; i++){
        cin >> scores[i];
    }
    // Print average
    cout << "Average: " << (scores[0] + scores[1] + scores[2]) / 3.0);
}
```



We can simplify or *abstract away* the calculation of the sum. Modify your code as follows:

```
1  #include<iostream>
2  using namespace std;
3
4  // constant
5  const int N = 3;
6
7  // prototype
8  int sum(int length, int array[]);
9
10 int main(void){
11     // Get scores
12     int scores[N];
13     for (int i = 0; i < N; i++) {
14         cout << "Score: ";
15         cin >> scores[i];
16     }
17     // Print sum
18     cout << "sum: " << sum(N, scores);
19 }
20
21 // calculate sum
22 int sum(int length, int array[]){
23     int sum = 0;
24     for (int i = 0; i < length; i++) {
25         sum += array[i];
26     }
27     return sum;
28 }
29
```

For previous example calculate the average of scores

Note:

- 1 Change N to be variable not constant (get from user)
- 2 Change sum function to avg function
- 3 The final result is float number with 2 number after decimal point



Write a program that takes scores from the user and print the numbers of scores greater than 10.

-- Note -- : you have to use an array! without array, the solution will be considered incorrect.

Example :

output >> Enter the number of elements :

input >> 5

output >> Enter element number 1 :

input >> 5

output >> Enter element number 2 :

input >> 1

output >> Enter element number 3 :

input >> 15

output >> Enter element number 4 :

input >> 2

output >> Enter element number 5 :

input >> 10

output >> number of scores greater than 10 : 1




```
1  #include<iostream>
2  using namespace std;
3
4  // constant
5  const int N = 3;
6
7  // prototype
8  int find(int length, int array[]);
9
10 int main(void){
11     int N;
12     cout << "Enter number of scores: ";
13     cin >> N;
14     // Get scores
15     int scores[N];
16     for (int i = 0; i < N; i++) {
17         cout << "Score: ";
18         cin >> scores[i];
19     }
20     // Print sum
21     cout << "number greater than 10: " << find(N, scores);
22 }
23
24 // calculate sum
25 int find(int length, int array[]){
26     int num = 0;
27     for (int i = 0; i < length; i++) {
28         if(array[i] > 10){
29             num += 1;
30         }
31     }
32     return num;
33 }
34
```



Variable Scope

Scope is a characterize of a variable that defines from which functions that variable may be accessed.

Local variables can only be accessed within the functions in which they are created.

Global variables can be accessed by any function in the program.



```

1  #include<iostream>
2  using namespace std;
3
4  // Global variable
5  int globalVar = 10;
6
7  // Function prototypes
8  void demonstrateLocalVariable();
9  void demonstrateGlobalVariable();
10
11 int main(void)
12 {
13     // Local variable in the main function
14     int localVar = 5;
15
16     cout << "Initial value of globalVar: " << globalVar << "\n";
17     cout << "Initial value of localVar in main: " << localVar << "\n";
18
19     // Calling functions that demonstrate local and global variable behavior
20     demonstrateLocalVariable();
21     demonstrateGlobalVariable();
22
23     // Showing that the global variable's value persists across functions
24     cout << "Value of globalVar after function calls: " << globalVar << "\n";
25     // Local variable only exists within this scope
26     cout << "Value of localVar in main after function calls: " << localVar << "\n";
27
28     return 0;
29 }
30
31 void demonstrateLocalVariable()
32 {
33     // Local variable within this function
34     int localVar = 20;
35     cout << "Local variable inside demonstrateLocalVariable: " << localVar << "\n";
36 }
37
38 void demonstrateGlobalVariable()
39 {
40     // Modify the global variable
41     globalVar = 50;
42     cout << "Global variable inside demonstrateGlobalVariable: " << globalVar << "\n";
43 }
44

```



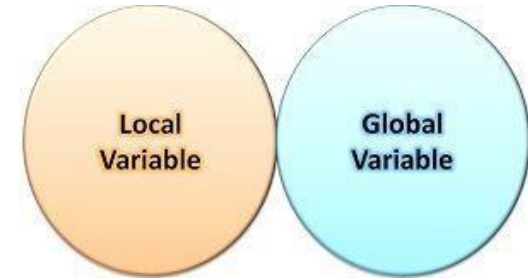
Explanation:

1. Global Variable (globalVar):

- Declared outside of any function (at the top of the program).
- Accessible and modifiable by all functions in the program.
- Its value persists across function calls.

2. Local Variable (localVar):

- Declared within a function, like main() or demonstrateLocalVariable().
- Only accessible within the function where it is declared.
- Each function can have its own localVar, but they are separate variables and do not interfere with each other.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  1

$ make variables
$ ./variables
Initial value of globalVar: 10
Initial value of localVar in main: 5
Local variable inside demonstrateLocalVariable: 20
Global variable inside demonstrateGlobalVariable: 50
Value of globalVar after function calls: 50
Value of localVar in main after function calls: 5
$
```

Key Points:

Local Variables: Variables declared within a function are local to that function. They can't be accessed or modified by other functions.

Global Variables: Variables declared outside of any function (globally) can be accessed and modified by any function within the program.

In this example:

`localVar` in `main()` and `localVar` in `demonstrateLocalVariable()` are different variables, despite having the same name, because they are local to their respective functions.

`globalVar` can be accessed and modified by any function, and changes to it persist across the program.





Support
Team

